Lesson 9: On the Scoreboard

Unity

VIDEO GAME CREATION

This week: track a real score, and show it - along with your lives - on screen, all the time, not just at the end.

Open your own [YourName]Game project from Lesson 8 - don't start a new one.

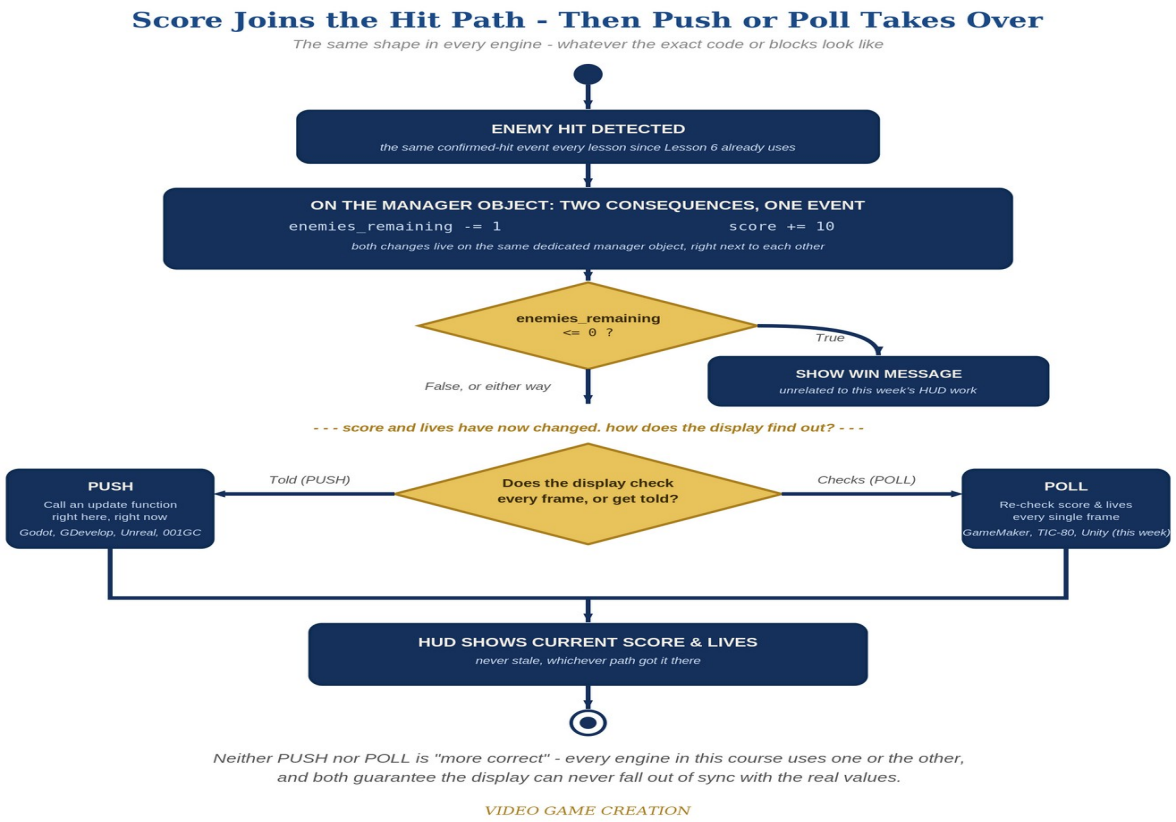
Tip: no new assets this week - this is a logic and UI lesson.

Setup

1. Open your [YourName]Game project from Lesson 8.
2. GameObject > UI (Canvas) > Legacy > Text, twice - rename them ScoreText and LivesText, and position them in a top corner (this automatically creates a Canvas if you don't have one yet).
3. Create an empty GameObject named HUD. Create a new C# script named HUD, attach it to this GameObject.

Level 1 - Minimal (MVP)

Destroying an enemy adds 10 points to your score. Score and lives are both visible on screen, all the time, not just at the start or end of the game.



Score joins the hit path, then Push or Poll takes over - the same shape in every engine, whatever the exact code or blocks look like.

4. In HUD.cs, add public Text scoreText and public Text livesText fields, then an Update() that continuously sets their text from GameManager.score and GameManager.lives (code below).
5. In the Inspector, drag your ScoreText and LivesText objects onto the HUD script's matching fields on the HUD GameObject.
6. In GameManager.cs, add public static int score = 0, and add score += 10 inside RegisterKill().
7. Press Play, shoot a few enemies, and let one escape - both texts should update immediately every time.

Highlighted lines are new this week; everything else is code you already have.

```
// HUD.cs (new script, attached to the HUD GameObject):
using UnityEngine;
using UnityEngine.UI;

public class HUD : MonoBehaviour {
    public Text scoreText;
    public Text livesText;

    void Update() {
        scoreText.text = "Score: " + GameManager.score;
        livesText.text = "Lives: " + GameManager.lives;
    }
}

// GameManager.cs - the new field and the revised RegisterKill():
```

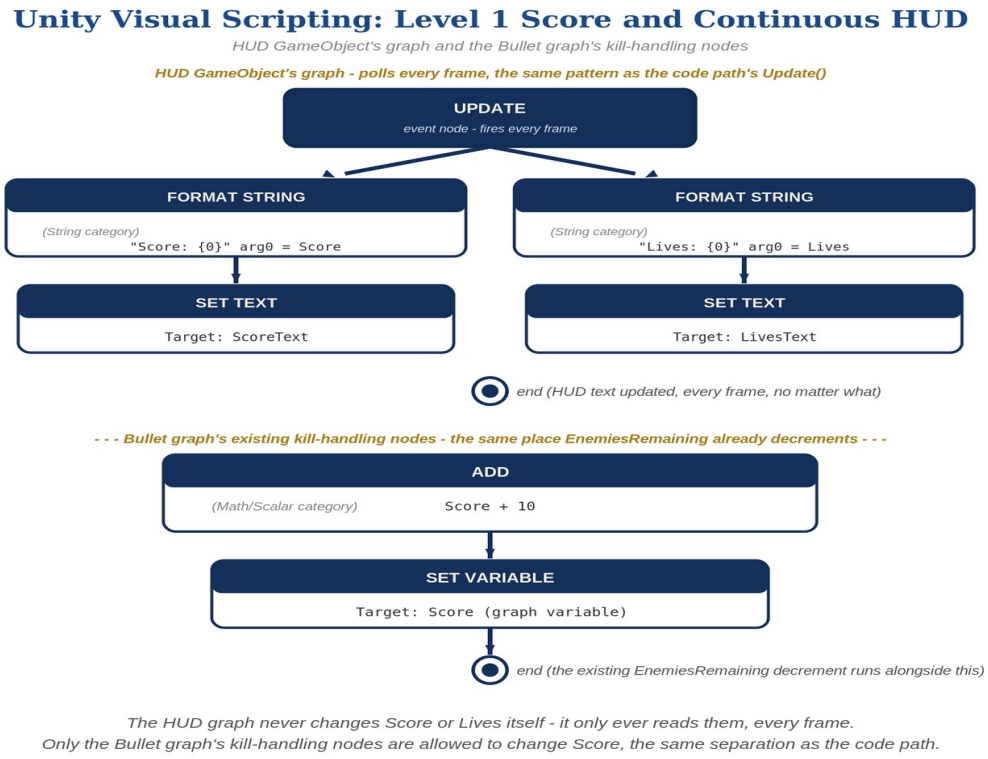
```
public static int score = 0;

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) hasWon = true;
}
```

Alternate Path: Visual Scripting

The same poll-every-frame HUD pattern works in a Script Graph instead of C#.

8. On the HUD GameObject's graph, add an Update event node feeding two "Format String" nodes (String category) - one building "Score: {0}" with the Score graph variable as its argument, one building "Lives: {0}" with Lives - each feeding its own "Set Text" node (one per Text object), the same poll-every-frame shape as the code path's Update().
9. On the Bullet graph's existing kill-handling nodes - the same place the EnemiesRemaining decrement already happens - add an "Add" node (Math/Scalar category) taking the Score graph variable and 10, feeding a "Set Variable" node targeting Score.



The HUD GameObject's graph and the Bullet graph's kill-handling nodes - the node-for-node equivalent of the Level 1 code.

Level 2 - Complete (Alpha/Beta)

The score and lives readouts never show a stale or wrong number - they always match reality, immediately.

- Shoot 3 enemies and confirm ScoreText reads exactly "Score: 30".
- Let an enemy escape and confirm ScoreText does NOT change - only LivesText should move.
- Because HUD.cs's Update() re-reads the values every single frame, the display can never go stale - unlike a system that only updates on specific events, there's nothing to forget to refresh.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: adjust font size/color on the Text (Legacy) components in the Inspector for a more game-like HUD look.
- Visual: change points-per-kill to Random.Range(5, 16) for scoring variety.
- Structure: HUD.cs's Update() polls (checks every frame, no matter what), while GameManager's RegisterKill()/RegisterEscape() are event-driven (only run when something happens). Both are valid ways to keep a display in sync - discuss with your instructor when each style fits better.

If Something's Wrong

- NullReferenceException on Play: confirm you actually dragged ScoreText and LivesText onto the HUD script's fields in the Inspector - an unassigned public field is the most common cause.
- Score doesn't appear correctly: confirm public static int score is spelled and cased exactly the same everywhere it's referenced.
- Text is invisible: confirm the Text (Legacy) objects' Rect Transform positions are actually within the Canvas/screen bounds, and that their color isn't the same as the background.

Want More?

Student Resource List > Unity section, or learn.unity.com